

```
hooking; the daemon exits */
return 0;
}
```

The hooks work with the following additional code:

```
// Preprocessor defines
#define malloc(A)
malloc(A, __FILE__, __FUNCTION__,
__LINE__)
#define realloc(A, B)
realloc(A, B, __FILE__, __FUNCTION__,
__LINE__)
#define free(A)
free(A, __FILE__, __FUNCTION__,
__LINE__)

void open_logs() {
hooks_active = 1;
...
}

// Intermediate function which decides
to go via our malloc hook or directly
to _libc_function
void *malloc(size_t size, const char
*file, const char *fn, const int ln) {
// Caller pointer
void *caller = __builtin_return__
address(0);

if (hooks_active) {
return my_malloc_hook(size, caller, file,
fn, ln);
}
return _libc_malloc(size);
}

// Malloc hook
void *my_malloc_hook(size_t size, void
*caller, const char *file, const char
*fn, const int ln) {
void *result;

// Deactivate hooks for logging
hooks_active = 0;
// Call real libc malloc
result = _libc_malloc(size);

// Do logging
printf("Memhook-malloc: sz=%d,
caller=%u, result=%u, file=%s, func=%s,
line=%d\n",
size, (unsigned int)caller, (unsigned
int)result, file, fn, ln); /* Prints on
the console */
```

```
...<To kernel module by means of ioctl>
// Reactivate hooks
hooks_active = 1;

return result;
}
```

When `hooks_active = 1`, the control flow will be: `Malloc() -> malloc_() -> my_malloc_hook() -> _libc_malloc() (-> pmm_ioctl() to kernel module).`

When `hooks_active` variable is zero, it will be: `Malloc() -> malloc_() -> _libc_malloc()`.

Kernel module. The `pmm` is used as a kernel module. The module `init()` registers the character device, say, `pmmdev`, and its fops (`open`, `ioctl`, `close`) structure. The `pmm_ioctl()` is the main contact point for the module, which has following ioctls:

PMM_RECORD_LOG `ioctl` for recording the memory request from hook functions. On receipt of `malloc` or `realloc` logs, these will be added to the log list. And, on receipt of free log, it will be compared with already logged `malloc` logs and, when a match is found, both `malloc` and free logs will be removed from the log list. (Because, the matching pair of `malloc` and free will not increase the memory footprint of a process.)

PMM_SET_CONFIG `ioctl` for starting or stopping logging for a particular user process.

PMM_GET_CONFIG `ioctl` for getting current configuration (kernel thread-execution cycle, etc).

PMM_MONITORED_PIDS for getting the list of monitored user processes.

Initialise proc entry (say, `/proc/pmm`) for user interaction. Our proc has two entries.

Cfg. Displays the module's configuration. This is used to start or stop scanning the kernel task structure by our kernel thread and modifying its frequency.

Pid. Used to display logs (memory-allocation logs sent from the user space daemon) to the user.



Spot an Error

And WIN Award!

You can win one thousand rupees plus a certificate from the editor of Electronics For You magazine if you spot and report any factual error in the latest issue of the magazine. Your report will be published in a coming issue of the magazine, giving credit to you.

Though every care is taken in preparation of the content for the magazine, an error can slip in at times. So we invite our learned readers to point out any error that they spot, for the benefit of the other readers.

Rules:

- The error has to be reported before end of the month that is mentioned on cover of the magazine
- The report has to be sent in writing or by email addressed to the Editor, EFY Magazine (editsec@efy.in)
- The error should be of factual nature, in the information published in the magazine, and not a typographical, grammatical or syntax error
- The correct information should also be provided along with the error spotted by you
- In case more than one person reports the same error, the one whose report reaches us the earliest will get the award money
- In case two or more persons jointly report an error, the award money will be divided amongst them
- EFY employees, their family members and friends, and EFY associates and their family members and friends are not eligible
- Decision of the editor will be final in this matter

electronics
FOR YOU